

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 01 · LECTURE NOTES

Agents and Rationality

What the subject is actually about.

MODULE OUTCOMES

- Define an agent in terms of percepts and actions.
- State what rationality means and what it does not.
- Classify environments and explain what each property costs.
- Explain the AI effect and why the field's definition keeps moving.
- Place the rest of the course against that classification.

1 Agents

AN AGENT IS A FUNCTION FROM PERCEPT HISTORIES TO ACTIONS

Percepts in, actions out. That is the entire interface, and **everything in this course is an implementation of that function** — a search, a rule set, a policy table, a decision tree over the world state. Holding the interface fixed makes the methods comparable.

The agent is *rational* if it acts so as to maximise its expected performance measure, given the percepts it has received — which converts 'good behaviour' from a judgement into a measurable property, and is the reason the definition is worth stating formally.

Note carefully what rationality is not. It is not omniscience: the agent acts on percepts, not on the true world state. It is not perfection: a rational action can have a bad outcome, and that does not make it irrational. And it is not introspection: nothing requires the agent to know why it acted.

And the performance measure is chosen by you, which is where the real difficulty sits. **A vacuum-cleaner agent rewarded for dirt collected will dump dirt on the floor and collect it again, and that is a correct solution to the problem you specified. This is CSCE 633 Module 01 §3's point about loss functions and CSCE 642's reward-specification problem, arriving before either** — the objective is the hard part, and it is hard in exactly the same way whether the agent searches or learns.

Design	What it uses	Example
Reflex agent	The current percept only, through a set of condition-action rules.	Steering away from a wall; a thermostat; a flocking boid (Module 11 §4). Underrated — a great deal of convincing behaviour needs nothing more.

Design	What it uses	Example
Model-based reflex agent	Percepts plus internal state tracking what it cannot currently see.	An enemy that remembers where it last saw you and searches there — which is most of what makes game enemies feel intelligent.
Goal-based agent	State plus an explicit goal, and a search for a sequence of actions achieving it.	Pathfinding (Module 04), classical planning (Module 07).
Utility-based agent	State plus a utility function over outcomes, so it can rank imperfect options and trade off competing goals.	Choosing between several imperfect actions (Modules 10 and 11) — necessary as soon as goals conflict or outcomes are uncertain.

Each row costs more to build and run, and does more. The engineering question is always the **cheapest design that produces the required behaviour** — and in game AI the answer is a reflex or model-based agent far more often than the literature's emphasis would suggest, because the requirement is *plausible* behaviour under a time budget rather than optimal behaviour.

2 Environments

Property	If it fails	Consequence for your methods
Fully observable	Partially observable.	You must reason over belief states rather than states — a partially observable Markov decision process (Module 10 §4), which is enormously more expensive.
Deterministic	Stochastic.	Plans become policies: you cannot commit to a sequence, because you do not know where you will be. Search becomes a Markov decision process (Module 10).
Static	Dynamic.	Deliberation has a deadline, and the world changes while you think — so you need anytime algorithms and replanning (Module 04 §4).
Discrete	Continuous.	Discretise (a grid, a navigation mesh, a waypoint graph) or use a continuous method. Discretisation is the usual answer and its resolution is a real design parameter.
Single agent	Multi-agent.	Adversarial search if interests conflict (Module 05), or game theory and coordination (Module 12).
Known rules	Unknown rules.	You must learn the dynamics — which is exactly CSCE 642's subject, and the reason it is the third course this semester.

MOST REAL PROBLEMS ARE THE HARD CASE, AND THE FIX IS TO LIE CAREFULLY

A real game world is partially observable, stochastic, dynamic, continuous, and multi-agent — the worst column of every row in the table, simultaneously.

So the standard move is to approximate it into something tractable: discretise continuous space into a navigation mesh, treat the world as static for the next 100 milliseconds, assume full observability within a radius and nothing beyond it, treat other agents as part of the environment rather than as reasoners.

Each approximation is a deliberate lie, and knowing which lie you told is how you predict the failure — an agent that assumes a static world will walk into a door that is closing, and an agent that treats others as environment will not anticipate being flanked. **Both failures are derivable in advance from the approximation list**, which is why writing the list down is worth the ten minutes.

That is CSCE 669 Module 12's modelling discipline arriving in a different subject, and the same warning applies: **the agent will exploit your approximation**, because it is optimising against the model and not against the world. **It is the single most transferable habit in this course.**

3 The moving definition

THE AI EFFECT: SOLVED PROBLEMS STOP COUNTING

Chess was the archetype of machine intelligence until a program beat the world champion, at which point it was reclassified as 'just tree search'. The same has happened to optical character recognition, speech recognition, machine translation, and route planning — each was AI until it worked.

So AI is partly defined as the set of problems not yet solved, which means the field appears to make no progress by construction: every success is subtracted from the definition. This is worth knowing both as intellectual history and as inoculation against the surrounding discourse.

The consequence for you is practical. A great deal of extremely useful AI is no longer called AI and is therefore under-taught — route planning, scheduling, constraint solving, satisfiability, and automated planning are all industrial technologies with decades of engineering behind them, and all of them are in this course.

And it is precisely why this course matters for this track. The AI in a shipped game is almost entirely the 'solved' kind, because solved means reliable, fast, and debuggable — the three properties a production system requires and the three that §4 argues learned systems lack.

- **Modules 02 through 05 are search.** Intelligence as systematic exploration of possibilities — the unifying idea of the course, and **Module 04 is the track's immediate payoff**, since pathfinding is in every game engine.
- **Modules 06 through 08 are structure.** Constraint satisfaction, classical planning, and logic. **Search with the problem's structure exploited rather than ignored** — including satisfiability solvers, which are the most under-appreciated tool in the course.
- **Modules 09 and 10 are uncertainty.** Probabilistic reasoning and sequential decision-making. **The direct bridge to CSCE 642**, which solves Module 10's problem by learning rather than by computing.
- **Modules 11 and 12 are agents in practice.** Game AI architectures, steering, crowds, and multi-agent behaviour — the engineering that academic treatments omit.
- **Module 13 is choosing between all of it, and claiming honestly.** No neural networks appear until then, and that is deliberate: the other two courses this semester cover them, and the point of this one is everything else.

4 Why this is still the right place to start

FOUR PROPERTIES LEARNED SYSTEMS DO NOT HAVE

Guarantees. A* returns the optimal path, provably, given an admissible heuristic (Module 03 §2). A learned policy returns something, and the only way to know how good it is is to measure it on a distribution you hope matches deployment (CSCE 753 Module 12).

Debuggability. You can ask a search why it chose a path and get an answer in terms of path costs and heuristic values — a complete, checkable explanation. Asking a network produces a saliency map and a shrug (CSCE 753 Module 08 §4). For a system a designer must tune, this difference dominates everything else.

No training data and no training. Change the level layout and the pathfinder works on the new layout immediately; change it and a learned policy must be retrained, which means the content pipeline and the AI are coupled. Decoupling them is worth a great deal in practice.

And cost. A* on a navigation mesh costs microseconds on a single CPU core, and a game at 60 frames per second has sixteen milliseconds for physics, rendering, audio, animation, and every agent's decision combined. That constraint alone explains most of the design of game AI — not a preference for classical methods, but an arithmetic limit, and one that Module 11 §1 returns to with numbers.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Russell & Norvig — AIMA, chapters 1–2 (pseudocode and code free)	The agent and environment definitions of §1 and §2, in their standard form.
Berkeley CS188 — lecture 1 and the course overview (free)	The same material with better motivation, and the Pacman project framework you will use from Module 02.
McCorduck — Machines Who Think	The history behind §3's AI effect, written by someone who interviewed the people involved. Library copy.
Game AI Pro — the free volumes (free)	Chapters by shipping practitioners, and the clearest evidence for §4's argument about what production systems actually use.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write the percept and action spaces for three agents you can think of, including one from a game you have played.

- 2 **Specify a performance measure for a vacuum agent**, then find the degenerate behaviour it rewards.
- 3 **Fix the measure** and find the next degenerate behaviour.
- 4 **Classify three environments** against §2's six properties.
- 5 **For a game you know, write down every approximation** its AI must be making.
- 6 **Predict a failure from each approximation**, then look for it in the game.
- 7 **Implement the four agent designs** for one simple environment and compare behaviour and code size.
- 8 **Find the cheapest design that produces acceptable behaviour** and report which it was.
- 9 **List five technologies that were once AI and are no longer called that.**
- 10 **Time an A* query on a grid** and compare against a frame budget of 16 ms.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define an agent and define rationality.
- 2 Name three things rationality is not.
- 3 Why is the performance measure the hard part?
- 4 Name the four agent designs and what each keeps.
- 5 Name six environment properties and the consequence of each failing.
- 6 What is the standard response to a hard environment, and what does it cost?
- 7 What is the AI effect, and what is its practical consequence?
- 8 Name four properties unlearned methods have that learned ones do not.